

TUẦN 5: Authentication & Security - Tài Liệu Lý Thuyết Chi Tiết

Giáo viên hướng dẫn: Senior Java Developer Cấp độ: Nâng cao (Tiếp theo Tuần 1-4) Thời lượng: 1 tuần (7 phiên)

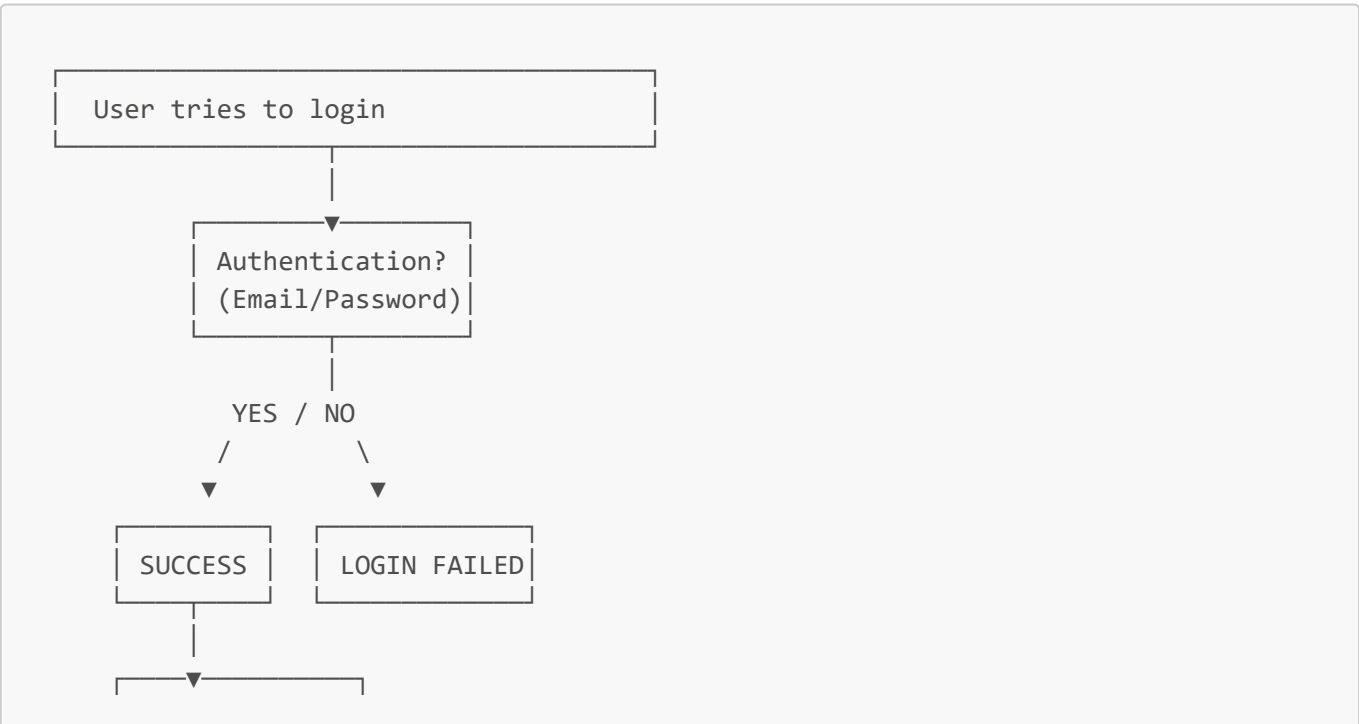
Nội dung Tuần 5

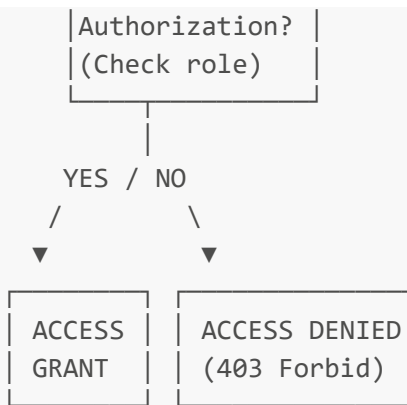
- **Session 29:** Auth Basics (Session-based Authentication)
- **Session 30:** Login/Register Flow
- **Session 31:** Role-based Authorization
- **Session 32:** Security Essentials
- **Session 33:** Session Timeout & Logout
- **Session 34:** Remember Me Feature
- **Session 35:** Mini Project

SESSION 29: Auth Basics (Session-based Authentication)

29.1 Authentication vs Authorization

Khái niệm	Định nghĩa	Ví dụ
Authentication (Xác thực)	Xác minh danh tính người dùng	Đăng nhập bằng email/password
Authorization (Phân quyền)	Xác minh quyền hạn người dùng	Admin xem được trang admin

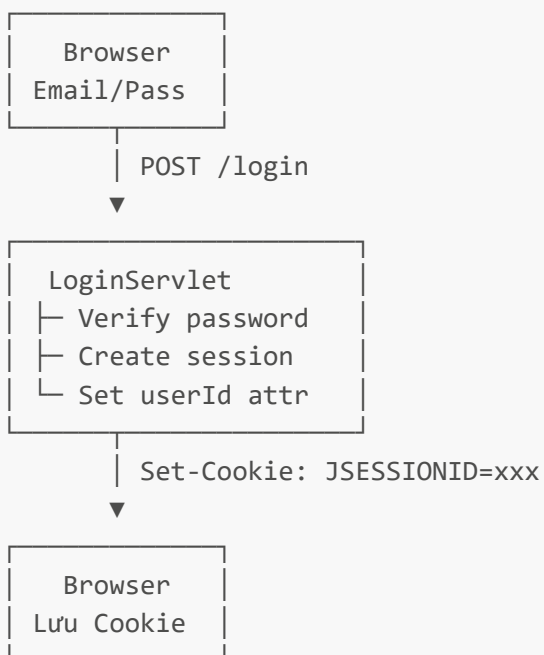




29.2 Session-based Authentication

Session = Một phiên làm việc của người dùng trên server.

STEP 1: User Login



STEP 2: Subsequent Requests



▼

Browser
View page

29.3 User Model

```
package com.example.model;

public class User {
    private int id;
    private String username;
    private String email;
    private String password_hash;
    private String role; // "user", "admin", "moderator"
    private boolean is_active;
    private String created_at;

    // Constructors
    public User() {}

    public User(String username, String email, String passwordHash, String role) {
        this.username = username;
        this.email = email;
        this.password_hash = passwordHash;
        this.role = role;
        this.is_active = true;
    }

    // Getters & Setters
    public int getId() { return id; }
    public void setId(int id) { this.id = id; }

    public String getUsername() { return username; }
    public void setUsername(String username) { this.username = username; }

    public String getEmail() { return email; }
    public void setEmail(String email) { this.email = email; }

    public String getPasswordHash() { return password_hash; }
    public void setPasswordHash(String hash) { this.password_hash = hash; }

    public String getRole() { return role; }
    public void setRole(String role) { this.role = role; }

    public boolean isActive() { return is_active; }
    public void setActive(boolean active) { this.is_active = active; }

    public String getCreatedAt() { return created_at; }
```

```
    public void setCreatedAt(String createdAt) { this.created_at = createdAt; }  
}
```

29.4 User DAO

```
package com.example.dao;  
  
import com.example.model.User;  
  
public interface UserDao {  
  
    /**  
     * Lấy user theo email  
     */  
    User findByEmail(String email) throws Exception;  
  
    /**  
     * Lấy user theo username  
     */  
    User findByUsername(String username) throws Exception;  
  
    /**  
     * Lấy user theo ID  
     */  
    User findById(int id) throws Exception;  
  
    /**  
     * Thêm user mới  
     */  
    int insert(User user) throws Exception;  
  
    /**  
     * Cập nhật user  
     */  
    boolean update(User user) throws Exception;  
  
    /**  
     * Kiểm tra email đã tồn tại  
     */  
    boolean isEmailExists(String email) throws Exception;  
  
    /**  
     * Kiểm tra username đã tồn tại  
     */  
    boolean isUsernameExists(String username) throws Exception;  
}
```

29.5 Login Servlet

```
@WebServlet("/login")
public class LoginServlet extends HttpServlet {

    private UserDao userDao = new UserDaoImpl();

    protected void doGet(HttpServletRequest request,
        HttpServletResponse response)
        throws ServletException, IOException {

        // Nếu đã login → redirect home
        HttpSession session = request.getSession();
        if (session.getAttribute("userId") != null) {
            response.sendRedirect("/app/home");
            return;
        }

        // Hiển thị login form
        request.getRequestDispatcher("login.jsp").forward(request, response);
    }

    protected void doPost(HttpServletRequest request,
        HttpServletResponse response)
        throws ServletException, IOException {

        String email = request.getParameter("email");
        String password = request.getParameter("password");

        // Validation
        if (email == null || email.trim().isEmpty()) {
            request.setAttribute("error", "Email is required");
            request.getRequestDispatcher("login.jsp").forward(request, response);
            return;
        }

        if (password == null || password.isEmpty()) {
            request.setAttribute("error", "Password is required");
            request.getRequestDispatcher("login.jsp").forward(request, response);
            return;
        }

        try {
            // Lấy user từ database
            User user = userDao.findByEmail(email.trim());

            if (user == null) {
                request.setAttribute("error", "Email or password incorrect");
                request.getRequestDispatcher("login.jsp").forward(request,
response);
                return;
            }

            // Check password using BCrypt
```

```

        if (!BCrypt.checkpw(password, user.getPasswordHash())) {
            request.setAttribute("error", "Email or password incorrect");
            request.getRequestDispatcher("login.jsp").forward(request,
response);
            return;
        }

        // Check if account active
        if (!user.isActive()) {
            request.setAttribute("error", "Account has been deactivated");
            request.getRequestDispatcher("login.jsp").forward(request,
response);
            return;
        }

        // ☒ Login success → Create session
        HttpSession session = request.getSession();
        session.setAttribute("userId", user.getId());
        session.setAttribute("username", user.getUsername());
        session.setAttribute("email", user.getEmail());
        session.setAttribute("role", user.getRole());

        // Set session timeout (30 minutes)
        session.setMaxInactiveInterval(30 * 60);

        // Redirect to home
        response.sendRedirect("/app/home");

    } catch (Exception e) {
        request.setAttribute("error", "Database error: " + e.getMessage());
        request.getRequestDispatcher("login.jsp").forward(request, response);
    }
}
}

```

29.6 Login Form (login.jsp)

```

<%@ page language="java" contentType="text/html; charset=UTF-8" pageEncoding="UTF-8" %>

<!DOCTYPE html>
<html>
<head>
    <title>Login</title>
    <style>
        * { margin: 0; padding: 0; box-sizing: border-box; }
        body { font-family: Arial; background: linear-gradient(135deg, #667eea 0%,
#764ba2 100%); height: 100vh; display: flex; justify-content: center; align-items:
center; }
        .login-container { background: white; padding: 40px; border-radius: 8px;

```

```

box-shadow: 0 8px 16px rgba(0,0,0,0.2); width: 100%; max-width: 400px; }
h1 { text-align: center; color: #333; margin-bottom: 30px; }
.form-group { margin-bottom: 20px; }
label { display: block; margin-bottom: 8px; color: #555; font-weight:
bold; }
input { width: 100%; padding: 10px; border: 1px solid #ddd; border-radius:
4px; font-size: 14px; }
input:focus { outline: none; border-color: #667eea; box-shadow: 0 0 5px
rgba(102, 126, 234, 0.3); }
.error { background: #f8d7da; color: #721c24; padding: 10px; border-
radius: 4px; margin-bottom: 20px; }
button { width: 100%; padding: 10px; background: #667eea; color: white;
border: none; border-radius: 4px; font-size: 16px; cursor: pointer; }
button:hover { background: #5568d3; }
.register-link { text-align: center; margin-top: 20px; font-size: 14px; }
.register-link a { color: #667eea; text-decoration: none; }
</style>
</head>
<body>
<div class="login-container">
<h1><img alt="lock icon" data-bbox="225 390 245 405"/> Login</h1>

<% if (request.getAttribute("error") != null) { %>
<div class="error">
<%= request.getAttribute("error") %>
</div>
<% } %>

<form method="POST" action="/app/login">
<div class="form-group">
<label for="email">Email:</label>
<input type="email" id="email" name="email" required value="<%=
request.getParameter("email") != null ? request.getParameter("email") : "" %>">
</div>

<div class="form-group">
<label for="password">Password:</label>
<input type="password" id="password" name="password" required>
</div>

<button type="submit">Login</button>
</form>

<div class="register-link">
Don't have an account? <a href="/app/register">Register here</a>
</div>
</div>
</body>
</html>

```

Session-based Auth:

- ☒ Xác thực = Verify danh tính
- ☒ Phân quyền = Verify rights
- ☒ Session lưu userId trên server
- ☒ Cookie (JSESSIONID) gửi cho browser

SESSION 30: Login/Register Flow & Password Hashing

30.1 BCrypt - Password Hashing Library

BCrypt = Thư viện mã hóa password an toàn.

Setup: pom.xml

```
<dependency>
  <groupId>org.mindrot</groupId>
  <artifactId>jbcrypt</artifactId>
  <version>0.4</version>
</dependency>
```

30.2 Hash Password

```
import org.mindrot.jbcrypt.BCrypt;

// Hash password
String password = "myPassword123";
String hashed = BCrypt.hashpw(password, BCrypt.gensalt());
// Result: $2a$10$BIX...

// Verify password
boolean isCorrect = BCrypt.checkpw("myPassword123", hashed);
// Result: true

boolean isWrong = BCrypt.checkpw("wrongPassword", hashed);
// Result: false
```

30.3 Khi nào nên hash password?

✗ SAI: Lưu plain text password
password: "myPassword123"

Database bị hack → nguy hiểm

☑ ĐÚNG: Lưu hashed password
password_hash: "\$2a\$10\$BIX/1e0YcF4G6SuS8R9jBejwZx5T2q8n6R..."

Database bị hack → không sao
(Không thể reverse hash)

30.4 Register Servlet

```
@WebServlet("/register")
public class RegisterServlet extends HttpServlet {

    private UserDao userDao = new UserDaoImpl();

    protected void doGet(HttpServletRequest request,
                          HttpServletResponse response)
        throws ServletException, IOException {

        // Nếu đã login → redirect home
        HttpSession session = request.getSession();
        if (session.getAttribute("userId") != null) {
            response.sendRedirect("/app/home");
            return;
        }

        request.getRequestDispatcher("register.jsp").forward(request, response);
    }

    protected void doPost(HttpServletRequest request,
                          HttpServletResponse response)
        throws ServletException, IOException {

        String username = request.getParameter("username");
        String email = request.getParameter("email");
        String password = request.getParameter("password");
        String confirmPassword = request.getParameter("confirmPassword");

        Map<String, String> errors = new HashMap<>();

        // Validation
        if (username == null || username.trim().isEmpty()) {
            errors.put("username", "Username is required");
        } else if (username.length() < 4) {
```

```
        errors.put("username", "Username must be at least 4 characters");
    }

    if (email == null || email.trim().isEmpty()) {
        errors.put("email", "Email is required");
    } else if (!email.matches("^[A-Za-z0-9+_.-]+@(.+)$")) {
        errors.put("email", "Invalid email format");
    }

    if (password == null || password.isEmpty()) {
        errors.put("password", "Password is required");
    } else if (password.length() < 6) {
        errors.put("password", "Password must be at least 6 characters");
    }

    if (!password.equals(confirmPassword)) {
        errors.put("confirmPassword", "Passwords do not match");
    }

    // Nếu có lỗi → quay lại form
    if (!errors.isEmpty()) {
        request.setAttribute("errors", errors);
        request.setAttribute("username", username);
        request.setAttribute("email", email);
        request.getRequestDispatcher("register.jsp").forward(request,
response);
        return;
    }

    try {
        // Check email đã tồn tại
        if (userDAO.isEmailExists(email.trim())) {
            request.setAttribute("error", "Email already registered");
            request.getRequestDispatcher("register.jsp").forward(request,
response);
            return;
        }

        // Check username đã tồn tại
        if (userDAO.isUsernameExists(username.trim())) {
            request.setAttribute("error", "Username already taken");
            request.getRequestDispatcher("register.jsp").forward(request,
response);
            return;
        }

        // ☒ Register success → Hash password & lưu
        String hashedPassword = BCrypt.hashpw(password, BCrypt.gensalt());

        User newUser = new User(
            username.trim(),
            email.trim(),
            hashedPassword,
            "user" // Default role
        );
    }
```

```

    );

    int newId = userDao.insert(newUser);

    // Redirect to login
    response.sendRedirect("/app/login?success=1");

    } catch (Exception e) {
        request.setAttribute("error", "Registration failed: " +
e.getMessage());
        request.getRequestDispatcher("register.jsp").forward(request,
response);
    }
}
}

```

30.5 Register Form (register.jsp)

```

<%@ page language="java" contentType="text/html; charset=UTF-8" pageEncoding="UTF-
8" %>
<%@ taglib prefix="c" uri="http://java.sun.com/jsp/jstl/core" %>

<!DOCTYPE html>
<html>
<head>
    <title>Register</title>
    <style>
        * { margin: 0; padding: 0; box-sizing: border-box; }
        body { font-family: Arial; background: linear-gradient(135deg, #667eea 0%,
#764ba2 100%); min-height: 100vh; padding: 20px; }
        .register-container { background: white; padding: 40px; border-radius:
8px; box-shadow: 0 8px 16px rgba(0,0,0,0.2); width: 100%; max-width: 400px;
margin: 0 auto; }
        h1 { text-align: center; color: #333; margin-bottom: 30px; }
        .form-group { margin-bottom: 20px; }
        label { display: block; margin-bottom: 8px; color: #555; font-weight:
bold; }
        input { width: 100%; padding: 10px; border: 1px solid #ddd; border-radius:
4px; }
        .error { background: #f8d7da; color: #721c24; padding: 10px; border-
radius: 4px; margin-bottom: 20px; }
        .field-error { color: #dc3545; font-size: 12px; margin-top: 5px; }
        button { width: 100%; padding: 10px; background: #667eea; color: white;
border: none; border-radius: 4px; cursor: pointer; }
        .login-link { text-align: center; margin-top: 20px; font-size: 14px; }
        .login-link a { color: #667eea; }
    </style>
</head>
<body>
    <div class="register-container">

```

```

<h1>📝 Register</h1>

<% if (request.getAttribute("error") != null) { %>
    <div class="error"><%= request.getAttribute("error") %></div>
<% } %>

<form method="POST" action="/app/register">
    <div class="form-group">
        <label for="username">Username:</label>
        <input type="text" id="username" name="username" required
            value="<%= request.getParameter("username") != null ?
request.getParameter("username") : "" %>">
        <c:if test="${ not empty errors.username }">
            <div class="field-error">${ errors.username }</div>
        </c:if>
    </div>

    <div class="form-group">
        <label for="email">Email:</label>
        <input type="email" id="email" name="email" required
            value="<%= request.getParameter("email") != null ?
request.getParameter("email") : "" %>">
        <c:if test="${ not empty errors.email }">
            <div class="field-error">${ errors.email }</div>
        </c:if>
    </div>

    <div class="form-group">
        <label for="password">Password:</label>
        <input type="password" id="password" name="password" required>
        <c:if test="${ not empty errors.password }">
            <div class="field-error">${ errors.password }</div>
        </c:if>
    </div>

    <div class="form-group">
        <label for="confirmPassword">Confirm Password:</label>
        <input type="password" id="confirmPassword" name="confirmPassword"
required>
        <c:if test="${ not empty errors.confirmPassword }">
            <div class="field-error">${ errors.confirmPassword }</div>
        </c:if>
    </div>

    <button type="submit">Register</button>
</form>

<div class="login-link">
    Already have an account? <a href="/app/login">Login here</a>
</div>
</div>
</body>
</html>

```

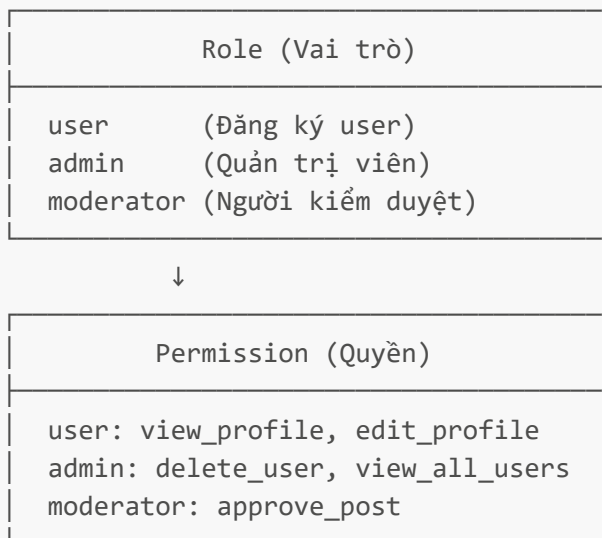
Tóm tắt Session 30

BCrypt & Register:

- ☒ **LUÔN** hash password trước lưu
- ☒ Không bao giờ lưu plain text password
- ☒ UserDao check email/username duplicate
- ☒ Validation đầy đủ (email format, password length)

SESSION 31: Role-based Authorization

31.1 Role vs Permission



31.2 AuthFilter - Kiểm tra Login

```
@WebFilter("/*")
public class AuthFilter implements Filter {

    private List<String> publicPages = Arrays.asList(
        "/login",
        "/register",
        "/css/",
        "/js/",
        "/images/"
    );

    @Override
    public void doFilter(ServletRequest request, ServletResponse response,
```

```

FilterChain chain)
    throws IOException, ServletException {

    HttpServletRequest req = (HttpServletRequest) request;
    HttpServletResponse res = (HttpServletResponse) response;

    String path =
req.getRequestURI().substring(req.getContextPath().length());

    // Nếu public page → cho qua
    if (isPublicPage(path)) {
        chain.doFilter(request, response);
        return;
    }

    // Nếu private page → check session
    HttpSession session = req.getSession(false);

    if (session == null || session.getAttribute("userId") == null) {
        // Chưa login → redirect login
        res.sendRedirect(req.getContextPath() + "/login");
        return;
    }

    // Đã login → cho qua
    chain.doFilter(request, response);
}

private boolean isPublicPage(String path) {
    return publicPages.stream().anyMatch(path::startsWith);
}

@Override
public void init(FilterConfig config) {}

@Override
public void destroy() {}
}

```

31.3 RoleFilter - Kiểm tra Role

```

@WebFilter("/*")
public class RoleFilter implements Filter {

    @Override
    public void doFilter(ServletRequest request, ServletResponse response,
FilterChain chain)
        throws IOException, ServletException {

        HttpServletRequest req = (HttpServletRequest) request;

```

```

        HttpServletResponse res = (HttpServletResponse) response;

        String path =
req.getRequestURI().substring(req.getContextPath().length());

        // Admin pages
        if (path.startsWith("/admin/")) {
            HttpSession session = req.getSession(false);

            if (session == null || !"admin".equals(session.getAttribute("role")))
{
                // Không phải admin → 403 Forbidden
                res.sendError(HttpServletResponse.SC_FORBIDDEN);
                return;
            }
        }

        chain.doFilter(request, response);
    }

    @Override
    public void init(FilterConfig config) {}

    @Override
    public void destroy() {}
}

```

web.xml

```

<?xml version="1.0" encoding="UTF-8"?>
<web-app version="4.0" xmlns="...">

    <filter>
        <filter-name>AuthFilter</filter-name>
        <filter-class>com.example.filter.AuthFilter</filter-class>
    </filter>
    <filter-mapping>
        <filter-name>AuthFilter</filter-name>
        <url-pattern>/*</url-pattern>
    </filter-mapping>

    <filter>
        <filter-name>RoleFilter</filter-name>
        <filter-class>com.example.filter.RoleFilter</filter-class>
    </filter>
    <filter-mapping>
        <filter-name>RoleFilter</filter-name>
        <url-pattern>/admin/*</url-pattern>
    </filter-mapping>

</web-app>

```

31.4 Admin Page Example

```
<%@ page language="java" contentType="text/html; charset=UTF-8" pageEncoding="UTF-8" %>

<!DOCTYPE html>
<html>
<head>
    <title>Admin Dashboard</title>
</head>
<body>
    <h1>👤 Admin Dashboard</h1>
    <p>Welcome, <%= session.getAttribute("username") %>!!</p>

    <ul>
        <li><a href="/app/admin/users">Manage Users</a></li>
        <li><a href="/app/admin/reports">View Reports</a></li>
        <li><a href="/app/admin/settings">Settings</a></li>
    </ul>
</body>
</html>
```

31.5 Check Role trong JSP

```
<%
    String role = (String) session.getAttribute("role");
    boolean isAdmin = "admin".equals(role);
%>

<h1>Users List</h1>

<% if (isAdmin) { %>
    <!-- Admin chỉ thấy nút Delete -->
    <table>
        <tr>
            <td>Name</td>
            <td><a href="delete">Delete</a></td>
        </tr>
    </table>
<% } else { %>
    <!-- Regular user chỉ thấy View -->
    <table>
        <tr>
            <td>Name</td>
            <td><a href="view">View</a></td>
        </tr>
    </table>
<% }
```

```
</table>
<% } %>
```

🔗 Tóm tắt Session 31

Role-based Auth:

- ☒ Role (admin, user, moderator)
- ☒ Filter kiểm tra login
- ☒ Filter kiểm tra role
- ☒ Conditional display trong JSP

🛡️ SESSION 32: Security Essentials

32.1 HTTPS (HTTP Secure)

HTTPS = Mã hóa communication giữa browser và server.

✗ HTTP (Not Secure)
Browser ↔ Server (plain text)
Password, data: người khác có thể xem!

☒ HTTPS (Secure)
Browser ↔ Server (Encrypted)
Password, data: Mã hóa, an toàn!

Enable HTTPS trên Tomcat

1. Generate SSL Certificate

```
keytool -genkey -alias tomcat -keyalg RSA -keystore keystore.jks -validity 365
```

2. Update `conf/server.xml`

```
<Connector port="8443" protocol="org.apache.coyote.http11.Http11NioProtocol"
    maxThreads="150" SSLEnabled="true" scheme="https" secure="true"
    keystoreFile="conf/keystore.jks" keystorePass="password"
    clientAuth="false" sslProtocol="TLS" />
```

3. Access: `https://localhost:8443/app`

32.2 Input Sanitization (Vệ sinh input)

XSS Prevention (Cross-Site Scripting)

```
// ✗ KHÔNG AN TOÀN
String userInput = "<script>alert('hacked')</script>";
out.print("Hello " + userInput); // XSS attack!

// ☑ AN TOÀN: Escape HTML
String escaped =
org.apache.commons.lang3.StringEscapeUtils.escapeHtml4(userInput);
out.print("Hello " + escaped); // Safe
```

HTML Escape

```
public class SecurityUtil {

    /**
     * Escape HTML special characters
     */
    public static String escapeHtml(String str) {
        if (str == null) return "";

        return str.replace("&", "&amp;")
            .replace("<", "&lt;")
            .replace(">", "&gt;")
            .replace("\"", "&quot;")
            .replace("'", "&#x27;");
    }
}
```

Trong JSP

```
<%- ✗ Không an toàn --%>
<p><%= userInput %></p>

<%- ☑ An toàn --%>
<p><c:out value="${ userInput }" /></p>
```

32.3 CSRF Protection (Cross-Site Request Forgery)

CSRF = Tấn công yêu cầu giả mạo.

```
Attacker Website (attacker.com)
↓

↓
Browser (đã login vào bank.com)
↓
Gửi request có cookie bank.com
↓
Bank bị "lừa" → transfer tiền!
```

Solution: CSRF Token

```
@WebServlet("/transfer")
public class TransferServlet extends HttpServlet {

    protected void doPost(HttpServletRequest request,
                           HttpServletResponse response)
        throws ServletException, IOException {

        // Get CSRF token từ form
        String tokenFromForm = request.getParameter("csrf_token");

        // Get CSRF token từ session
        String tokenFromSession = (String) request.getSession()
            .getAttribute("csrf_token");

        // Verify token
        if (!tokenFromForm.equals(tokenFromSession)) {
            response.sendError(HttpServletResponse.SC_FORBIDDEN, "CSRF token
invalid");
            return;
        }

        // Process transfer
        // ...
    }

    protected void doGet(HttpServletRequest request,
                           HttpServletResponse response)
        throws ServletException, IOException {

        // Generate CSRF token
        String token = UUID.randomUUID().toString();
        request.getSession().setAttribute("csrf_token", token);

        request.getRequestDispatcher("transfer.jsp").forward(request, response);
    }
}
```

transfer.jsp

```
<form method="POST" action="/app/transfer">
  <!-- Hidden CSRF token -->
  <input type="hidden" name="csrf_token" value="<%=
session.getAttribute("csrf_token") %>">

  <input type="text" name="amount" placeholder="Amount">
  <button type="submit">Transfer</button>
</form>
```

32.4 CORS (Cross-Origin Resource Sharing)

CORS = Cho phép request từ domain khác.

```
Domain 1: example.com
Domain 2: api.example.com

example.com gửi request tới api.example.com
↓
Browser block (CORS policy)
↓
Cần server api.example.com allow domain example.com
```

Allow CORS trong Servlet

```
@WebServlet("/api/data")
public class DataServlet extends HttpServlet {

    protected void doGet(HttpServletRequest request,
                          HttpServletResponse response)
        throws ServletException, IOException {

        // Allow CORS
        response.setHeader("Access-Control-Allow-Origin", "*");
        response.setHeader("Access-Control-Allow-Methods", "GET, POST, OPTIONS");
        response.setHeader("Access-Control-Allow-Headers", "Content-Type");

        response.setContentType("application/json");
        response.getWriter().print("{\"message\":\"Hello\"}");
    }
}
```

CORS Filter

```
@WebFilter("/*")
public class CORSFilter implements Filter {

    @Override
    public void doFilter(ServletRequest request, ServletResponse response,
        FilterChain chain)
        throws IOException, ServletException {

        HttpServletResponse res = (HttpServletResponse) response;

        res.setHeader("Access-Control-Allow-Origin", "https://example.com");
        res.setHeader("Access-Control-Allow-Methods", "GET, POST, PUT, DELETE,
OPTIONS");
        res.setHeader("Access-Control-Allow-Headers", "Content-Type,
Authorization");

        chain.doFilter(request, response);
    }

    @Override
    public void init(FilterConfig config) {}

    @Override
    public void destroy() {}
}
```

Tóm tắt Session 32

Security Essentials:

- ☒ HTTPS: Mã hóa communication
- ☒ Input Sanitization: Escape HTML
- ☒ CSRF Protection: Verify token
- ☒ CORS: Control cross-origin requests

SESSION 33: Session Timeout & Logout

33.1 Session Timeout

Set trong Servlet

```
HttpSession session = request.getSession();
session.setMaxInactiveInterval(30 * 60); // 30 minutes
```

Set trong web.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<web-app version="4.0" xmlns="...">

    <session-config>
        <cookie-config>
            <http-only>true</http-only>
            <secure>false</secure>
        </cookie-config>
        <tracking-mode>COOKIE</tracking-mode>
        <!-- Session timeout: 30 minutes -->
        <timeout>30</timeout>
    </session-config>

</web-app>
```

33.2 Logout Servlet

```
@WebServlet("/logout")
public class LogoutServlet extends HttpServlet {

    protected void doGet(HttpServletRequest request,
                          HttpServletResponse response)
        throws ServletException, IOException {

        HttpSession session = request.getSession(false);

        if (session != null) {
            // Invalidate session
            session.invalidate();
        }

        // Redirect to login
        response.sendRedirect("/app/login?logout=1");
    }
}
```

33.3 Session Listener

```
import javax.servlet.http.HttpSessionEvent;
import javax.servlet.http.HttpSessionListener;

@WebListener
public class SessionListener implements HttpSessionListener {

    @Override
```

```

    public void sessionCreated(HttpSessionEvent event) {
        System.out.println("☑ Session created: " + event.getSession().getId());
        System.out.println("    Active sessions: " + getActiveSessions());
    }

    @Override
    public void sessionDestroyed(HttpSessionEvent event) {
        System.out.println("✗ Session destroyed: " +
event.getSession().getId());
        System.out.println("    Active sessions: " + getActiveSessions());
    }

    private static int activeSessions = 0;

    private int getActiveSessions() {
        return activeSessions;
    }
}

```

33.4 Check Session Timeout

```

@WebFilter("/*")
public class SessionTimeoutFilter implements Filter {

    @Override
    public void doFilter(ServletRequest request, ServletResponse response,
FilterChain chain)
        throws IOException, ServletException {

        HttpServletRequest req = (HttpServletRequest) request;
        HttpServletResponse res = (HttpServletResponse) response;

        HttpSession session = req.getSession(false);

        if (session != null) {
            long lastAccessTime = session.getLastAccessedTime();
            long currentTime = System.currentTimeMillis();
            long timeout = session.getMaxInactiveInterval() * 1000;

            if (currentTime - lastAccessTime > timeout) {
                // Session expired
                session.invalidate();
                res.sendRedirect(req.getContextPath() + "/login?expired=1");
                return;
            }
        }

        chain.doFilter(request, response);
    }
}

```

```
@Override
public void init(FilterConfig config) {}

@Override
public void destroy() {}
}
```

Tóm tắt Session 33

Session Management:

- ☒ Set MaxInactiveInterval
- ☒ Logout = invalidate session
- ☒ Session listeners
- ☒ Check timeout

SESSION 34: Remember Me Feature (Optional)

34.1 Remember Me Cách Hoạt Động

```
STEP 1: User login + check "Remember Me"
  ↓
STEP 2: Server tạo persistent cookie (14 ngày)
  ↓
STEP 3: Browser lưu cookie
  ↓
STEP 4: Lần sau user quay lại
  ├── Cookie còn hết hạn?
  ├── YES → Auto-login
  └── NO → Requires login
```

34.2 Login với Remember Me

```
@WebServlet("/login")
public class LoginServlet extends HttpServlet {

    protected void doPost(HttpServletRequest request,
                          HttpServletResponse response)
        throws ServletException, IOException {

        String email = request.getParameter("email");
        String password = request.getParameter("password");
        String rememberMe = request.getParameter("rememberMe");
```

```

    try {
        User user = userDao.findByEmail(email.trim());

        if (user == null ||
            !BCrypt.checkpw(password, user.getPasswordHash())) {
            request.setAttribute("error", "Invalid credentials");
            request.getRequestDispatcher("login.jsp").forward(request,
response);
            return;
        }

        // Create session
        HttpSession session = request.getSession();
        session.setAttribute("userId", user.getId());
        session.setAttribute("username", user.getUsername());

        // If "Remember Me" checked
        if ("on".equals(rememberMe)) {
            // Tạo persistent token
            String token = generateRememberMeToken();

            // Lưu token vào database cho user này
            userDao.setRememberMeToken(user.getId(), token);

            // Tạo cookie (14 ngày)
            Cookie cookie = new Cookie("rememberMe", token);
            cookie.setMaxAge(14 * 24 * 60 * 60); // 14 days
            cookie.setHttpOnly(true);
            cookie.setSecure(true);
            response.addCookie(cookie);
        }

        response.sendRedirect("/app/home");

    } catch (Exception e) {
        request.setAttribute("error", "Error: " + e.getMessage());
        request.getRequestDispatcher("login.jsp").forward(request, response);
    }
}

private String generateRememberMeToken() {
    return UUID.randomUUID().toString() + System.currentTimeMillis();
}
}

```

34.3 Auto-login Filter

```

@WebFilter("/*")
public class AutoLoginFilter implements Filter {

```

```
private UserDAO userDAO = new UserDAOImpl();

@Override
public void doFilter(ServletRequest request, ServletResponse response,
FilterChain chain)
    throws IOException, ServletException {

    HttpServletRequest req = (HttpServletRequest) request;
    HttpServletResponse res = (HttpServletResponse) response;

    // Nếu đã login → skip
    HttpSession session = req.getSession(false);
    if (session != null && session.getAttribute("userId") != null) {
        chain.doFilter(request, response);
        return;
    }

    // Check Remember Me cookie
    Cookie[] cookies = req.getCookies();
    if (cookies != null) {
        for (Cookie cookie : cookies) {
            if ("rememberMe".equals(cookie.getName())) {
                String token = cookie.getValue();

                try {
                    // Verify token từ database
                    User user = userDAO.findByRememberMeToken(token);

                    if (user != null && user.isActive()) {
                        // Auto-login
                        session = req.getSession();
                        session.setAttribute("userId", user.getId());
                        session.setAttribute("username", user.getUsername());
                        session.setAttribute("email", user.getEmail());
                        session.setAttribute("role", user.getRole());
                    }
                } catch (Exception e) {
                    e.printStackTrace();
                }
            }
        }
    }

    chain.doFilter(request, response);
}

@Override
public void init(FilterConfig config) {}

@Override
public void destroy() {}
}
```

34.4 Login Form với Remember Me

```
<form method="POST" action="/app/login">
  <div class="form-group">
    <label for="email">Email:</label>
    <input type="email" id="email" name="email" required>
  </div>

  <div class="form-group">
    <label for="password">Password:</label>
    <input type="password" id="password" name="password" required>
  </div>

  <div class="form-group">
    <label>
      <input type="checkbox" name="rememberMe">
      Remember me for 14 days
    </label>
  </div>

  <button type="submit">Login</button>
</form>
```

Tóm tắt Session 34

Remember Me:

- ☒ Generate token unique
- ☒ Lưu token vào database
- ☒ Tạo persistent cookie (HttpOnly, Secure)
- ☒ Auto-login khi có cookie

SESSION 35: Mini Project - Auth System with Roles

35.1 Yêu cầu Dự Án

Tạo **Complete Authentication System**:

- Register user
- Login with hashed password
- Session management
- Role-based access (Admin/User)
- Remember Me
- Logout

- Admin dashboard
 - User profile
-

35.2 Database Schema

```
CREATE DATABASE auth_system;
USE auth_system;

-- Users table
CREATE TABLE users (
    id INT AUTO_INCREMENT PRIMARY KEY,
    username VARCHAR(50) UNIQUE NOT NULL,
    email VARCHAR(100) UNIQUE NOT NULL,
    password_hash VARCHAR(255) NOT NULL,
    role ENUM('user', 'admin') DEFAULT 'user',
    is_active BOOLEAN DEFAULT TRUE,
    remember_me_token VARCHAR(255),
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP
);

-- Audit log table
CREATE TABLE audit_logs (
    id INT AUTO_INCREMENT PRIMARY KEY,
    user_id INT,
    action VARCHAR(100) NOT NULL,
    timestamp TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE SET NULL
);

-- Sample data
INSERT INTO users (username, email, password_hash, role) VALUES
('admin', 'admin@example.com', '$2a$10$BIX/1e0YcF4G6SuS8R9jBejwZx5T2q8n6R...',
'admin'),
('user1', 'user1@example.com', '$2a$10$BIX/1e0YcF4G6SuS8R9jBejwZx5T2q8n6R...',
'user');
```

35.3 Project Structure

```
AuthSystemApp/
├─ src/main/java/com/example/
│   ├─ model/
│   │   └─ User.java
│   ├─ dao/
│   │   ├─ UserDAO.java
│   │   └─ UserDAOImpl.java
│   └─ servlet/
│       └─ LoginServlet.java
```

```
| | RegisterServlet.java
| | LogoutServlet.java
| | ProfileServlet.java
| | AdminServlet.java
| filter/
| | AuthFilter.java
| | RoleFilter.java
| | AutoLoginFilter.java
| | CORSFilter.java
| util/
| | DatabaseConnection.java
| | SecurityUtil.java
| | ValidationUtil.java
| listener/
| | SessionListener.java
|
| src/main/webapp/
| | WEB-INF/
| | | web.xml
| | | context.xml
| | views/
| | | login.jsp
| | | register.jsp
| | | home.jsp
| | | profile.jsp
| | | admin/
| | | | dashboard.jsp
| | | | users.jsp
| | | error/
| | | | 403.jsp
| | | | 404.jsp
| | css/
| | | style.css
| | js/
| | | validation.js
|
| lib/
| | mysql-connector-java.jar
| | commons-dbcp.jar
| | jbcrypt-0.4.jar
| | jstl-1.2.jar
```

35.4 User Model

```
package com.example.model;

public class User {
    private int id;
    private String username;
    private String email;
```

```

private String password_hash;
private String role; // "user" hoặc "admin"
private boolean is_active;
private String remember_me_token;
private String created_at;

// Constructors & Getters/Setters (giống Session 29)
// ...
}

```

35.5 UserDAO Interface

```

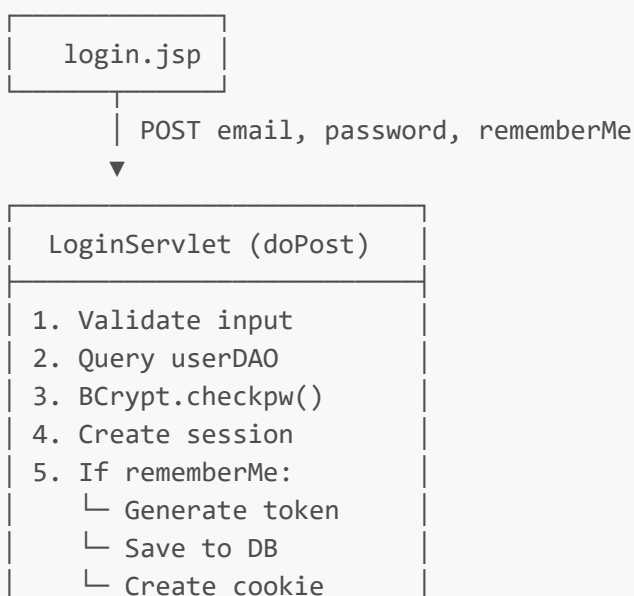
package com.example.dao;

import com.example.model.User;

public interface UserDAO {
    User findByEmail(String email) throws Exception;
    User findByUsername(String username) throws Exception;
    User findById(int id) throws Exception;
    User findByRememberMeToken(String token) throws Exception;
    int insert(User user) throws Exception;
    boolean update(User user) throws Exception;
    boolean isEmailExists(String email) throws Exception;
    boolean isUsernameExists(String username) throws Exception;
    void setRememberMeToken(int userId, String token) throws Exception;
    java.util.List<User> findAll() throws Exception;
}

```

35.6 Complete Login Flow (Diagram)



6. Log action (audit)
7. Redirect /home



home.jsp
(Protected)

35.7 Views - Dashboard Examples

home.jsp

```
<%@ page language="java" contentType="text/html; charset=UTF-8" pageEncoding="UTF-8" %>

<!DOCTYPE html>
<html>
<head>
    <title>Dashboard</title>
    <style>
        * { margin: 0; padding: 0; }
        body { font-family: Arial; background: #f5f5f5; }
        .navbar { background: #333; padding: 15px 20px; color: white; display:
flex; justify-content: space-between; }
        .container { max-width: 1200px; margin: 30px auto; background: white;
padding: 30px; border-radius: 8px; }
        .welcome { color: #333; }
        .btn { padding: 10px 20px; background: #007bff; color: white; border:
none; border-radius: 4px; cursor: pointer; }
        .admin-btn { background: #dc3545; }
        a { color: white; text-decoration: none; margin-left: 20px; }
    </style>
</head>
<body>
    <div class="navbar">
        <div class="welcome">
            👤 Welcome, <%= session.getAttribute("username") %>!
        </div>
        <div>
            <a href="/auth/profile">Profile</a>
            <% if ("admin".equals(session.getAttribute("role"))) { %>
                <a href="/auth/admin/dashboard" class="admin-btn">Admin Panel</a>
            <% } %>
            <a href="/auth/logout">Logout</a>
        </div>
    </div>

    <div class="container">
        <h1>🏠 Dashboard</h1>
```

```

    <p>This is a protected page. Only logged-in users can see this.</p>

    <h2>Your Profile</h2>
    <ul>
        <li>Username: <%= session.getAttribute("username") %></li>
        <li>Email: <%= session.getAttribute("email") %></li>
        <li>Role: <%= session.getAttribute("role") %></li>
    </ul>
</div>
</body>
</html>

```

admin/dashboard.jsp

```

<%@ page language="java" contentType="text/html; charset=UTF-8" pageEncoding="UTF-8" %>
<%@ taglib prefix="c" uri="http://java.sun.com/jsp/jstl/core" %>

<!DOCTYPE html>
<html>
<head>
    <title>Admin Dashboard</title>
    <style>
        /* Same as home.jsp */
    </style>
</head>
<body>
    <div class="navbar">
        <div>👤 Admin Dashboard</div>
        <div>
            <a href="/auth/home">Home</a>
            <a href="/auth/admin/users">Manage Users</a>
            <a href="/auth/logout">Logout</a>
        </div>
    </div>

    <div class="container">
        <h1>🔑 Admin Panel</h1>

        <h2>System Statistics</h2>
        <ul>
            <li>Total Users: ${totalUsers}</li>
            <li>Active Users: ${activeUsers}</li>
            <li>Admin Users: ${adminUsers}</li>
        </ul>

        <h2>Recent Activity</h2>
        <table style="width: 100%; border-collapse: collapse;">
            <thead>
                <tr style="background: #f0f0f0;">
                    <th style="padding: 10px; border: 1px solid #ddd;">User</th>

```

```

        <th style="padding: 10px; border: 1px solid #ddd;">Action</th>
        <th style="padding: 10px; border: 1px solid
#ddd;">Timestamp</th>
    </tr>
</thead>
<tbody>
    <c:forEach var="log" items="${logs}">
        <tr>
            <td style="padding: 10px; border: 1px solid
#ddd;">${log.username}</td>
            <td style="padding: 10px; border: 1px solid
#ddd;">${log.action}</td>
            <td style="padding: 10px; border: 1px solid
#ddd;">${log.timestamp}</td>
        </tr>
    </c:forEach>
</tbody>
</table>
</div>
</body>
</html>

```

35.8 Key Servlets

LoginServlet (Complete)

```

@WebServlet("/login")
public class LoginServlet extends HttpServlet {

    private UserDao userDao = new UserDaoImpl();

    protected void doPost(HttpServletRequest request, HttpServletResponse
response)
        throws ServletException, IOException {

        String email = request.getParameter("email");
        String password = request.getParameter("password");
        String rememberMe = request.getParameter("rememberMe");

        if (email == null || password == null) {
            request.setAttribute("error", "Email and password required");
            request.getRequestDispatcher("login.jsp").forward(request, response);
            return;
        }

        try {
            User user = userDao.findByEmail(email.trim());

            if (user == null || !BCrypt.checkpw(password, user.getPasswordHash()))
{

```

```
        request.setAttribute("error", "Invalid credentials");
        request.getRequestDispatcher("login.jsp").forward(request,
response);
        return;
    }

    if (!user.isActive()) {
        request.setAttribute("error", "Account has been deactivated");
        request.getRequestDispatcher("login.jsp").forward(request,
response);
        return;
    }

    // ☒ Create session
    HttpSession session = request.getSession();
    session.setAttribute("userId", user.getId());
    session.setAttribute("username", user.getUsername());
    session.setAttribute("email", user.getEmail());
    session.setAttribute("role", user.getRole());
    session.setMaxInactiveInterval(30 * 60);

    // Remember Me
    if ("on".equals(rememberMe)) {
        String token = UUID.randomUUID().toString();
        userDao.setRememberMeToken(user.getId(), token);

        Cookie cookie = new Cookie("rememberMe", token);
        cookie.setMaxAge(14 * 24 * 60 * 60);
        cookie.setHttpOnly(true);
        response.addCookie(cookie);
    }

    // Log audit
    logAction(user.getId(), "LOGIN");

    response.sendRedirect("/auth/home");

} catch (Exception e) {
    request.setAttribute("error", "Error: " + e.getMessage());
    request.getRequestDispatcher("login.jsp").forward(request, response);
}

}

private void logAction(int userId, String action) {
    try {
        // Insert into audit_logs table
        // ...
    } catch (Exception e) {
        e.printStackTrace();
    }
}

}
```

35.9 Deploy & Test

```
# 1. Create MySQL database
mysql -u root -p < auth_system.sql

# 2. Update DatabaseConnection.java
# 3. Build and deploy to Tomcat
# 4. Access http://localhost:8080/AuthSystemApp

# Test accounts:
# Email: admin@example.com, Password: admin123
# Email: user1@example.com, Password: user123
```

Hoàn thành Tuần 5!

Bạn đã học:

- ☒ Session-based Authentication
- ☒ Login/Register Flow
- ☒ Password Hashing (BCrypt)
- ☒ Role-based Authorization
- ☒ Security Essentials (HTTPS, XSS, CSRF, CORS)
- ☒ Session Management & Logout
- ☒ Remember Me Feature
- ☒ Mini Project - Complete Auth System

Bước tiếp theo: Tuần 6 - Advanced JSP/Servlet (Filters, Listeners, File Upload, i18n)

Tài liệu Tham Khảo

- OWASP Authentication Cheat Sheet
- BCrypt Documentation
- HTTP Security Best Practices
- Session Management Guidelines
- CSRF Prevention Techniques

Tài liệu được biên soạn bởi: Senior Java Developer Ngày cập nhật: 2025 Phiên bản: 1.0